Job Title	[The title of the position]	Position Type	Remote/Contractor
Department	[Name of the department where the role sits]	Line Manager	[The person or position to whom the role reports]

Material for reference before starting the task	https://docs.pears.com/
Task Duration	2 days

Task

Build a simplified GPU cloud service platform that demonstrates both your fullstack capabilities and understanding of distributed systems. The platform should provide API key management, model deployment, and inference capabilities similar to a mini AI PaaS.

You have been tasked with:

- Backend Infrastructure: Creating a Node.js backend that manages API keys, handles inference requests, and simulates workload distribution across multiple "worker nodes"
- Distributed Architecture: Implementing a basic distributed task queue or P2P communication mechanism for handling inference requests across simulated GPU workers
- Frontend Dashboard: Building a web interface for API key management, usage monitoring, and basic model deployment workflows
- API Design: Developing OpenAI-compatible inference endpoints that support both streaming and non-streaming responses
- Monitoring & Telemetry: Implementing basic usage tracking, rate limiting, and a simple metrics dashboard
- Documentation: Providing clear setup instructions, API documentation, and architectural decisions

Technology Stack & Implementation Choices

This task involves designing a **distributed GPU cloud service platform**, and the following technologies and implementation choices reflect real-world AI infrastructure patterns.

- Backend API Layer: The backend will be built using Node.js with Fastify as the primary web framework. Fastify (ref) is chosen for its high performance, low overhead, and strong plugin ecosystem, making it well-suited for handling high-throughput inference requests and API key management.
- Worker Communication Layer: For distributed communication between inference workers, the system will use Hyperswarm RPC (ref) or Hyperswarm (ref). This enables decentralized, peer-to-peer communication between worker nodes without relying on a central message broker. It also simplifies NAT traversal and allows dynamic worker discovery.
- Distributed Data Store : Instead of a traditional centralized database, the platform will use HyperDB as a distributed key-value store. HyperDB (ref) allows shared state (such as worker metadata, job states, and usage records) to be synchronized across nodes in a decentralized manner.

Submission Steps

Deliverables:

- Complete source code repository on GitHub with clear README and setup instructions
- Live demo URL (deployed version) - extra points for a working deployment
- API documentation (OpenAPI spec preferred)
- Brief architecture document (2-3 pages PDF) explaining:
    - System design and component interaction
    - Distributed architecture approach
    - Trade-offs and assumptions made
    - Scalability considerations
    - Areas for future improvement
- A video (maximum 5 minutes) explaining your code and the project, highlighting tradeoffs and assumptions made during the development process

GitHub Repository Requirements:

- Clear folder structure separating backend, frontend, and shared code (if any)
- Docker Compose setup for local development

- Environment configuration examples
- Basic test coverage for critical paths
- CI/CD pipeline configuration (GitHub Actions or similar)

Best Practices

For the Backend:

- Focus on clean, modular architecture that could realistically scale - align with the design patterns present in the reference repositories
- Implement proper error handling and input validation
- Use environment variables for configuration
- Include health check endpoints
- Demonstrate understanding of async patterns and proper resource management

For the Frontend:

- Create a functional, responsive interface (design quality is secondary to functionality)
- Implement proper state management
- Show real-time updates where appropriate (e.g., usage metrics)
- Include loading states and error handling

For the Distributed Components:

- You can simulate distributed behavior locally - we're more interested in the architecture than actual multi-machine deployment
- Document how your approach would scale in a real production environment
- Consider failure scenarios and implement basic resilience patterns

General Guidelines:

- Prioritize working functionality over extensive features
- Use AI tools for boilerplate and research, but ensure you understand and can explain every architectural decision
- Include comments for complex logic
- Keep the scope realistic - we prefer 2 features done well over 5 features done poorly

Evaluation Criteria

We are not expecting a production-ready platform. We want to see how you approach building complex distributed systems, make architectural decisions, and balance backend robustness with frontend usability. Your ability to explain your choices and identify limitations is as important as the implementation itself.